# HookChain

## A new perspective for Bypassing EDR Solutions

Helvio Junior (M4v3r1ck)

# Agenda

- Whoami
- What is HookChain?
- Research limitations
- Protection rings
- Windows APIs and process loading
- Function Hooks
- EDR overview
- Most common EDR bypass public techniques
- HookChain

# Whoami

- Helvio Junior (M4v3r1ck)
- Airplane pilot
- First to earn OSCE³ in Latin America
- Studying to OSEE
- Research topics:
  - Low Level Security
  - Buffer Overflow
  - Shellcoding
  - Malware development
  - AV/EDR/XDR Bypasses
  - Mobile and etc...
- CEO of Sec4US

# Motivation

- Many people believe, I have solution X, which is a Gartner leader, so I am 100% protected
- There is no silver bullet
- Bypass/avoidance of defense layers
- FUD - Fully UnDetectable
- FUD is when your code is not detected by any defense layer/tool
- It is important to be 100% FUD regarding your target

# What is HookChain?

- Is a Technique
- It is NOT a tool
- User-land NTDLL Hook bypass

# Limitations!

- This study aims to demonstrate a new bypass technique using the interception of the operating system API functions of Microsoft Windows© 64-bit in user mode.
- Limited just at Ntdll hooks bypass.
- This does NOT aim to classify the defense and EDR products demonstrated here in terms of their effectiveness, efficacy, and quality in the process of protecting and defending the assets where they are installed, as it is a study and presentation of a technique focused on a single point of identification of the agents.

# Ethics!

- This study does not represent ethical violations
- All tests were conducted in controlled environments with valid licensing
- I called several EDR/XDR vendors to have early access at this research and PoC, but no one answered
- https://www.linkedin.com/posts/helviojunior_hookchain-edrbypass-xdrbypass-activity-7188225698434596865-3Jxy
- The public released code is focused at HookChain itself

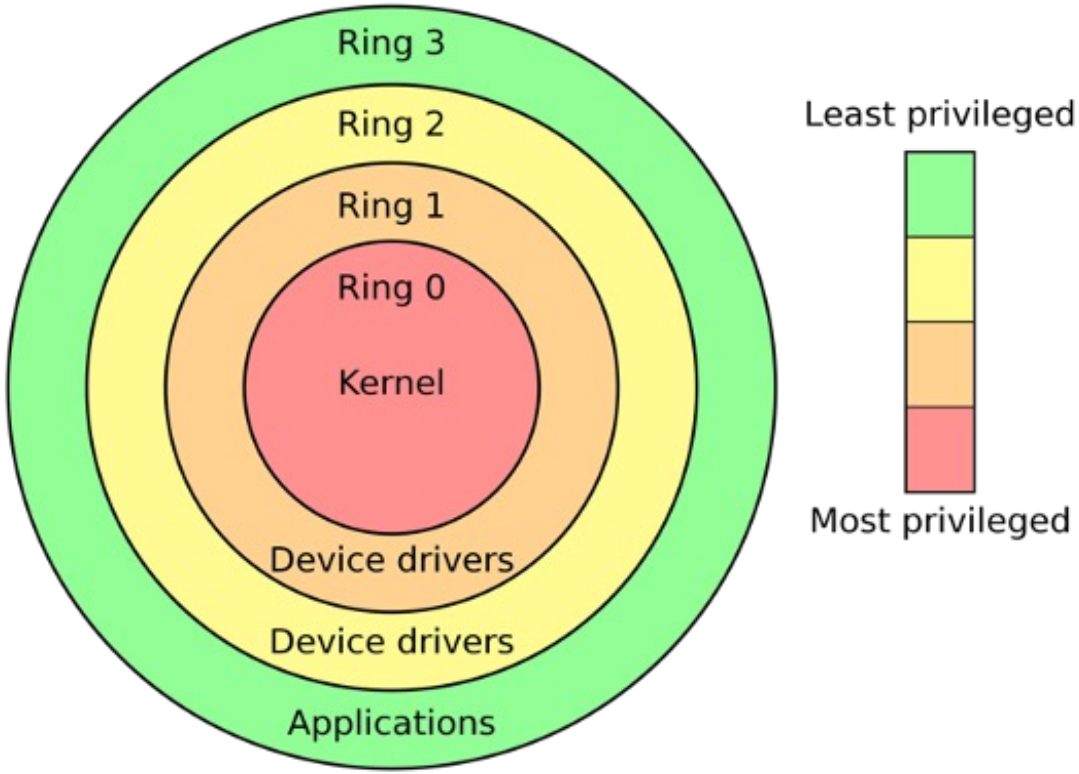
# Protection Rings

From a security point of view, there are protection rings

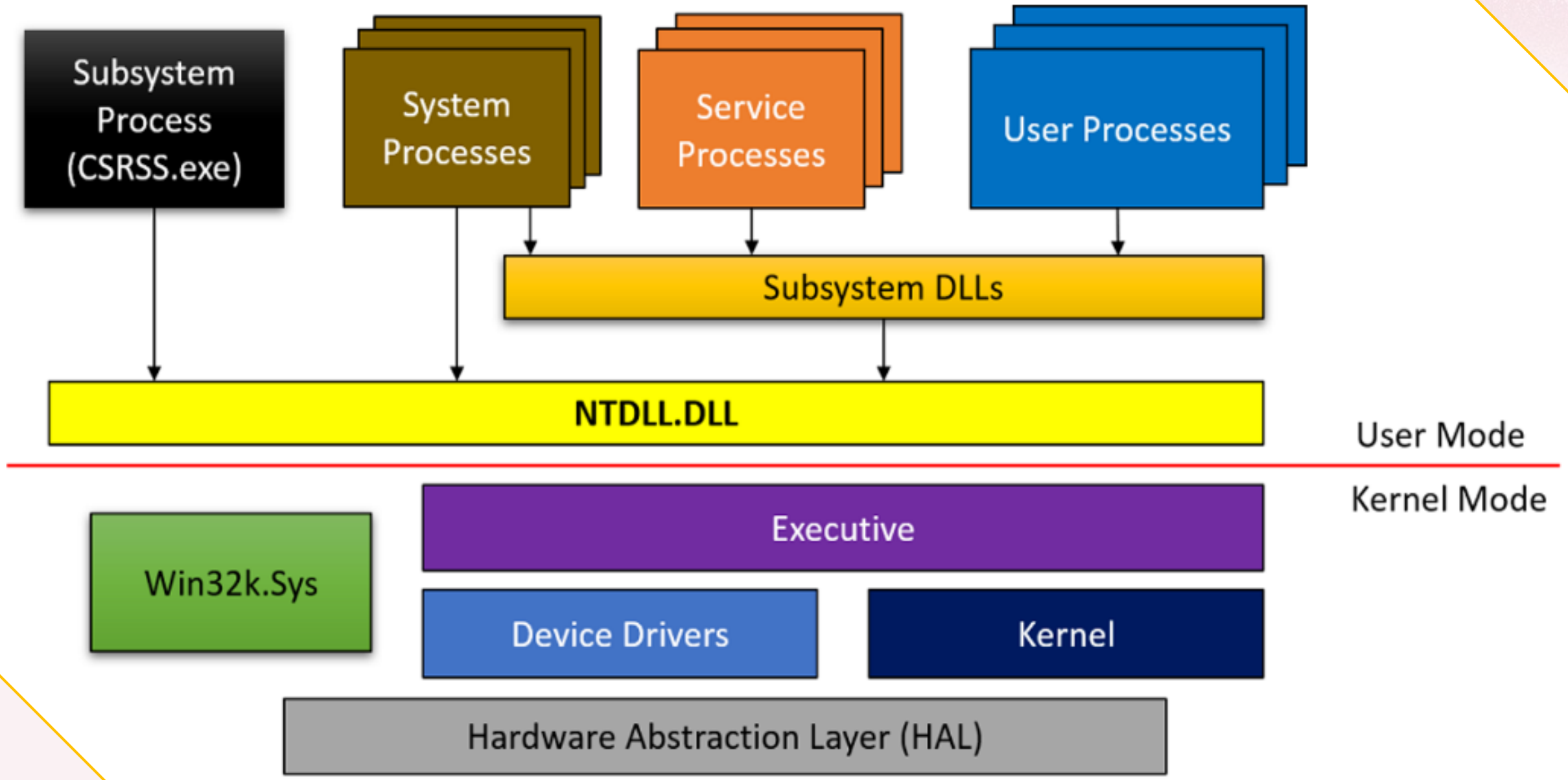
Aims to create protection mechanisms against failures and malicious actions

For today's topic, the main objective is to protect critical areas of the OS from applications executed by the user

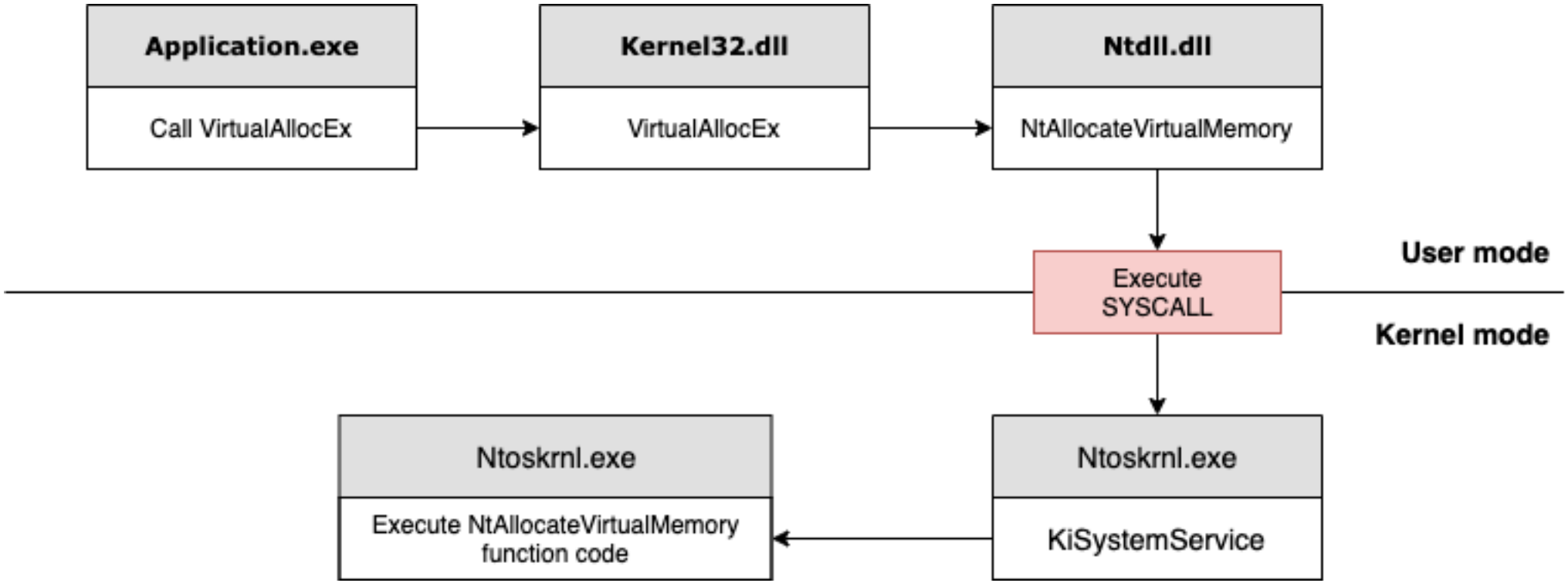
- Ring 3 (user mode/user land): Restricted access to resources and
- Ring 0 (kernel mode): Has direct access to resources such as memory, CPU, system, etc.



# Windows APIs

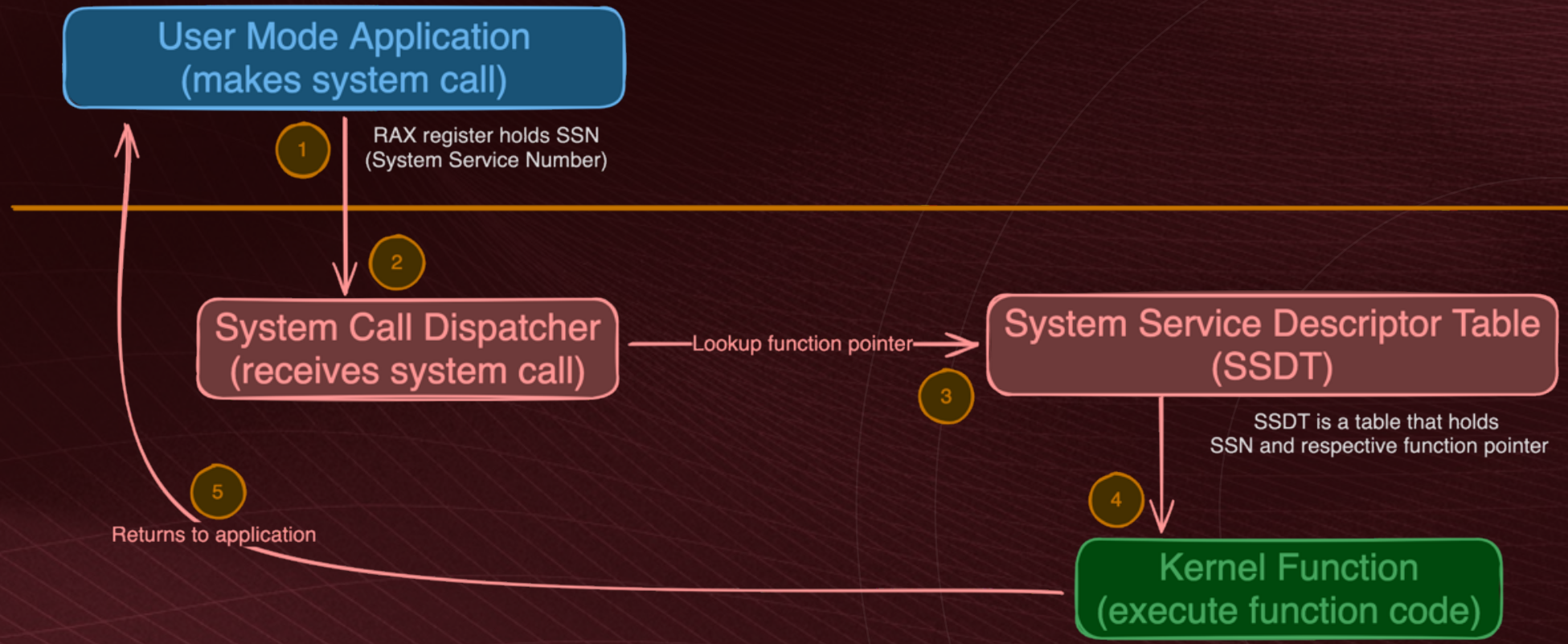


# Common call stack

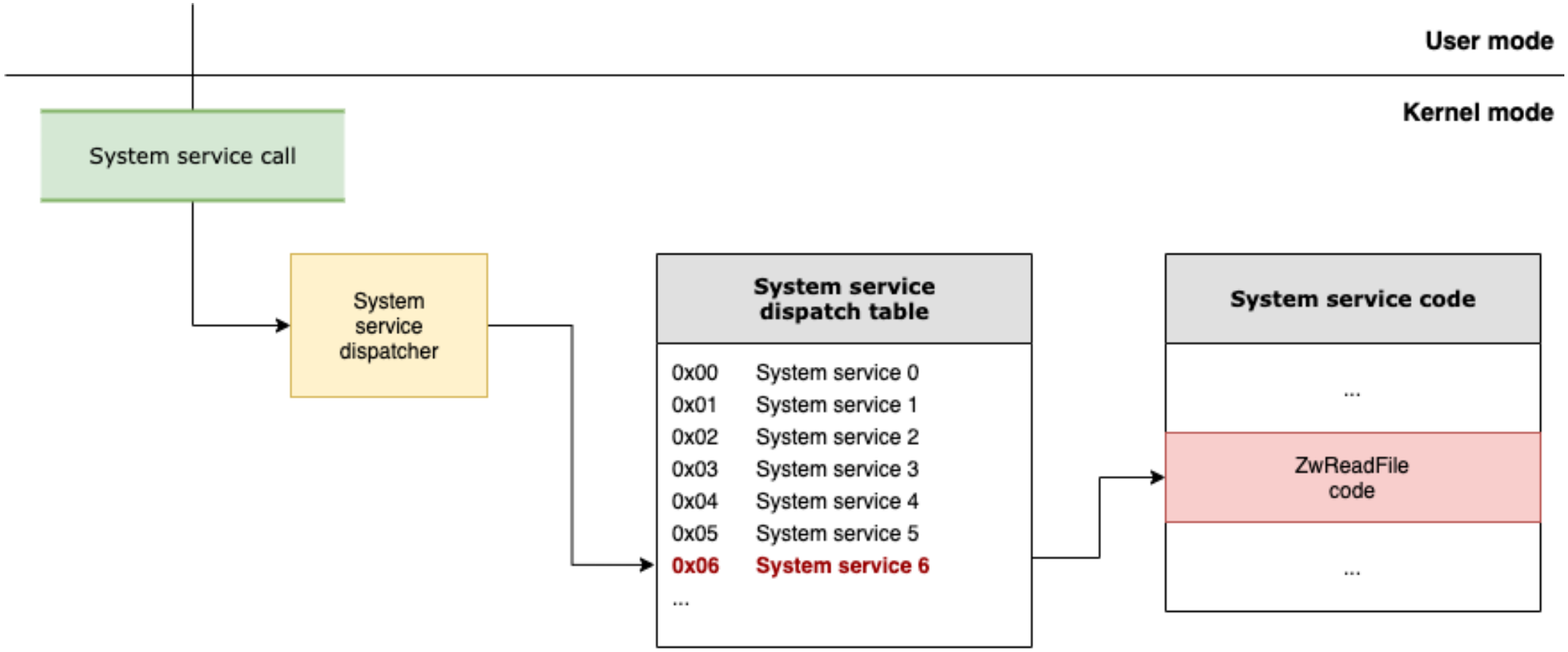


# Common execution flow

- The user mode application call **SYSCALL** assembly instruction
- The system call dispatcher uses the system call number (**SSN**) to look up the corresponding function pointer in the SSDT.
- The appropriate kernel function is called.
- After the kernel function completes, control is returned to the user mode application.



# Common execution flow



# Image Loader

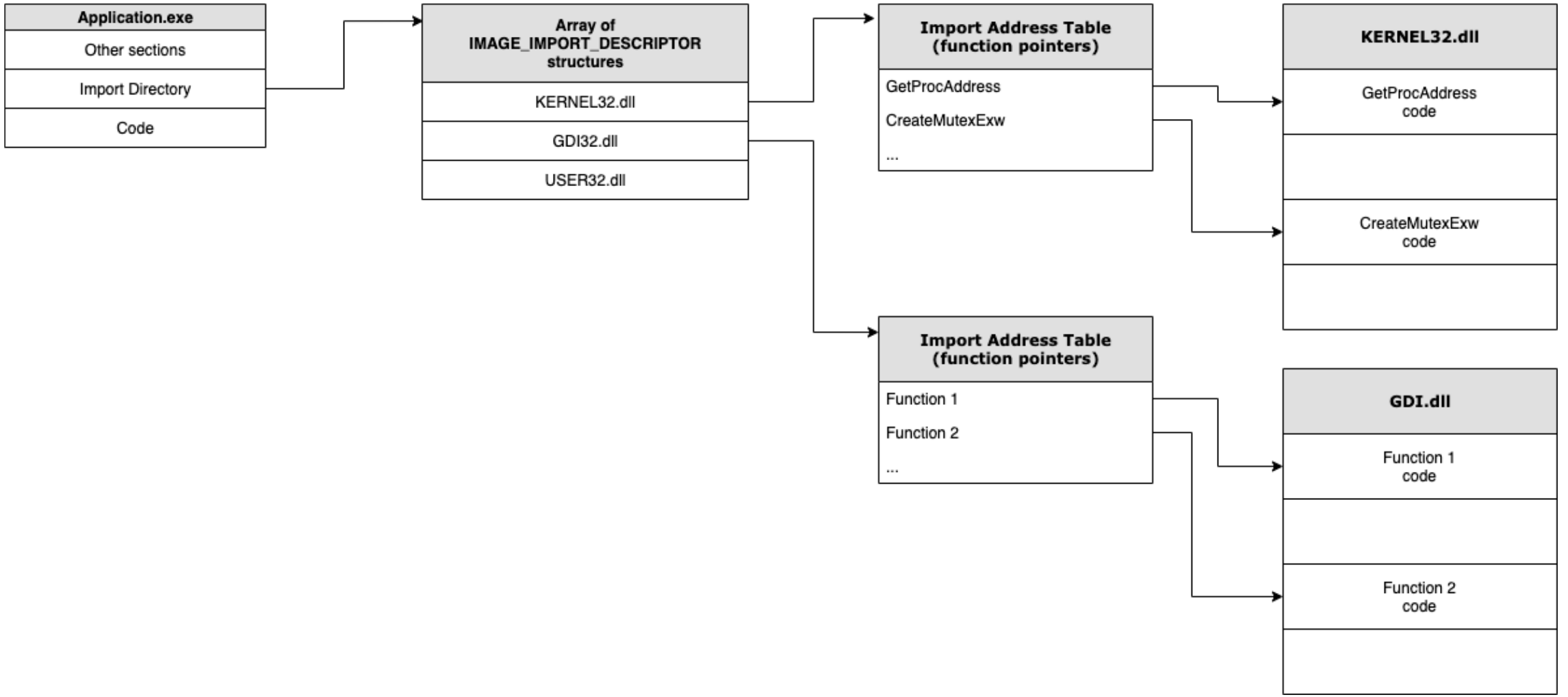
- When a process is initiated, or a new DLL have been loaded
- Several actions are carried out internally by the operating system
- The image loader  is a user-mode resident code, within Ntdll.dll and not in a kernel library
- Import Address Table (IAT) is filled with the current addresses of the function in memory

# Image Loader

## IAT - Import Address Table

| | |
|---|---|
| DLL | Loads each one of the DLLs referenced in the PE import table. |
| Previous loaded | Checks if the DLL in question is already loaded into the process's memory, if not, reads the DLL from disk and maps it into memory. |
| Recursive loading | After mapping the DLL into memory, this process is repeated for this DLL with the goal of importing the dependencies used by it. |
| Fill IAT with function pointer | After each DLL is loaded, the IAT is processed looking for the specific functions to be imported. For each imported name, the loader checks the export table of the imported DLL and tries to locate the desired function. |

# PE Import Schema



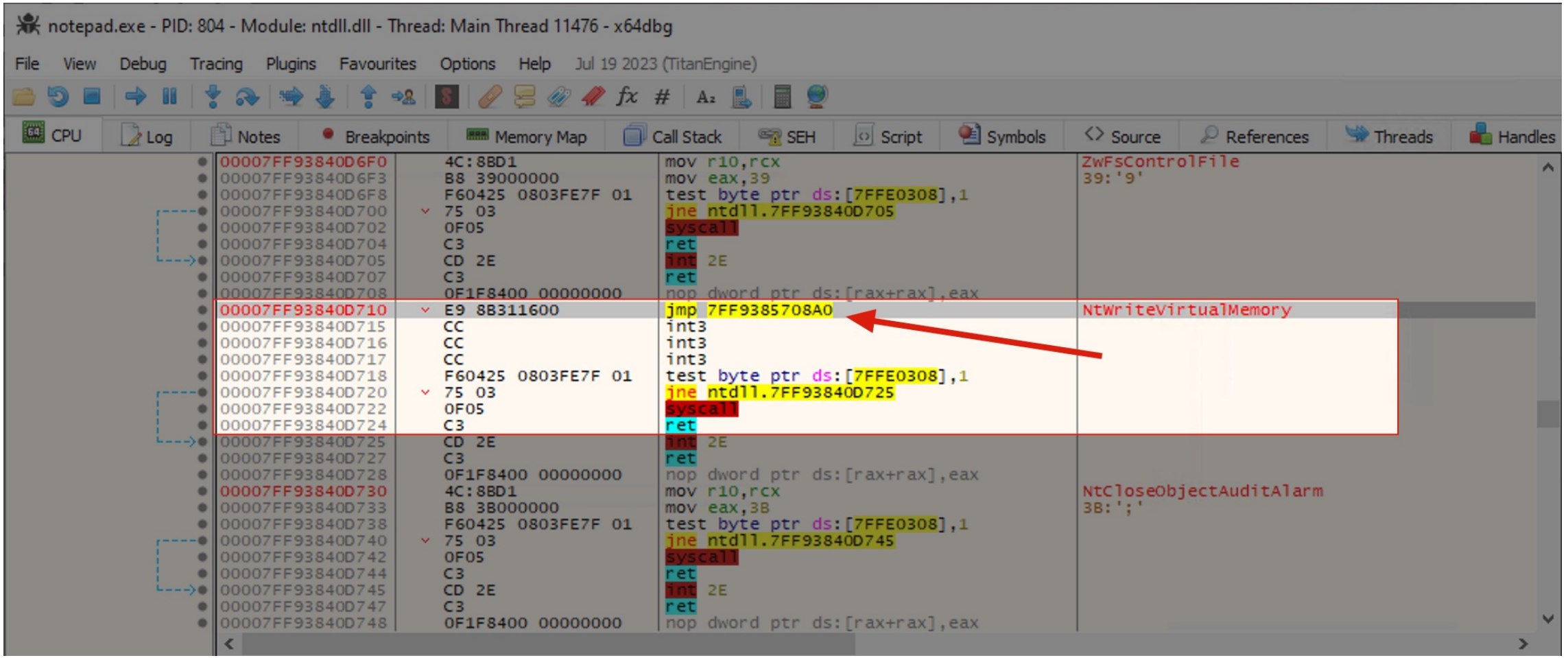
# Function Hook

- Function interception (Hook) is not something new and has various applications such as application debugging. But also, in the monitoring process by defense layers (EDR/XDR).

- Let’s take a look at 2 types:
  - JMP or Call
  - IAT Hook

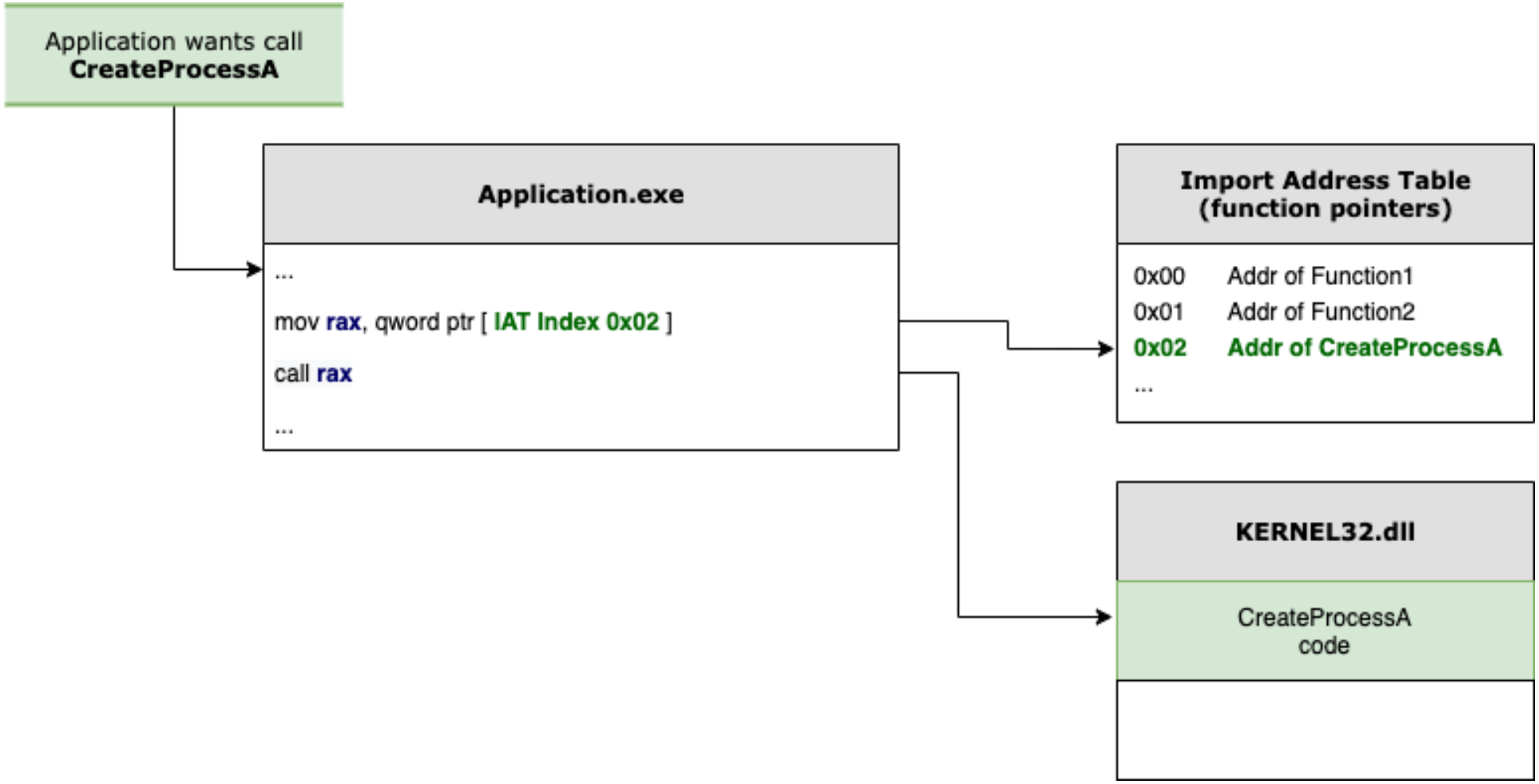
# Function Hook

## Ntdll JMP



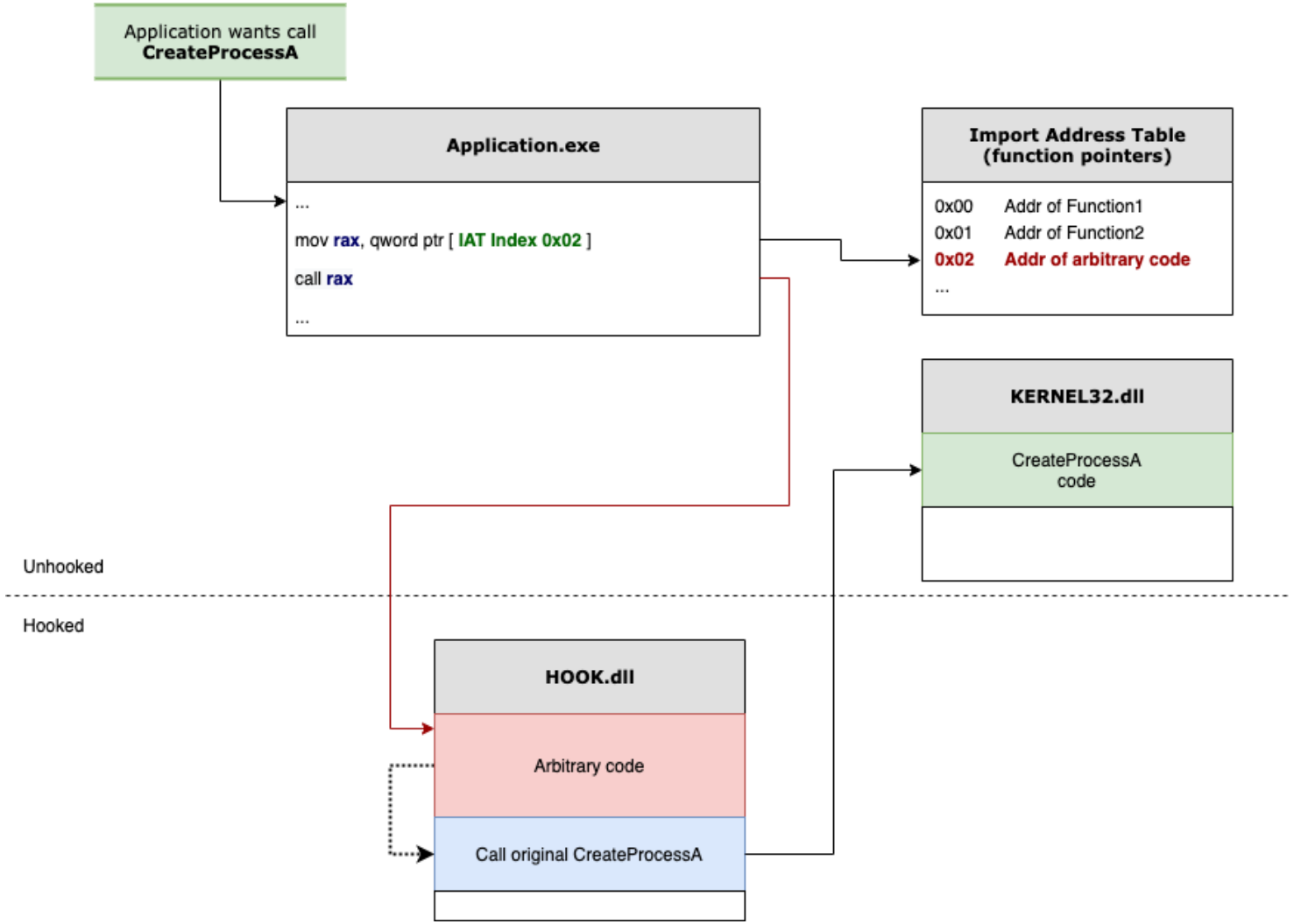
# Function Hook

## IAT Original/unhooked execution flow

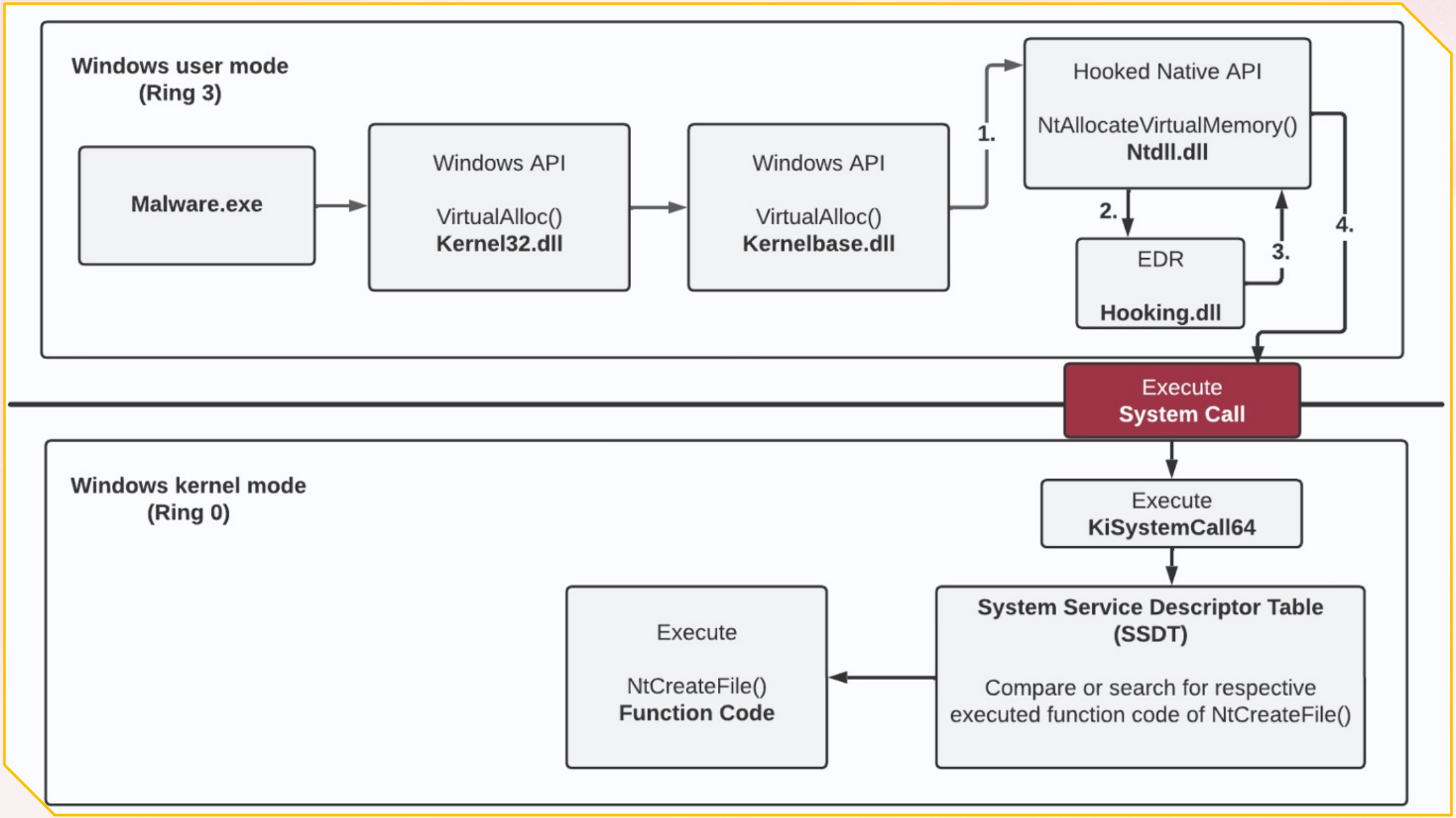


# Function Hook

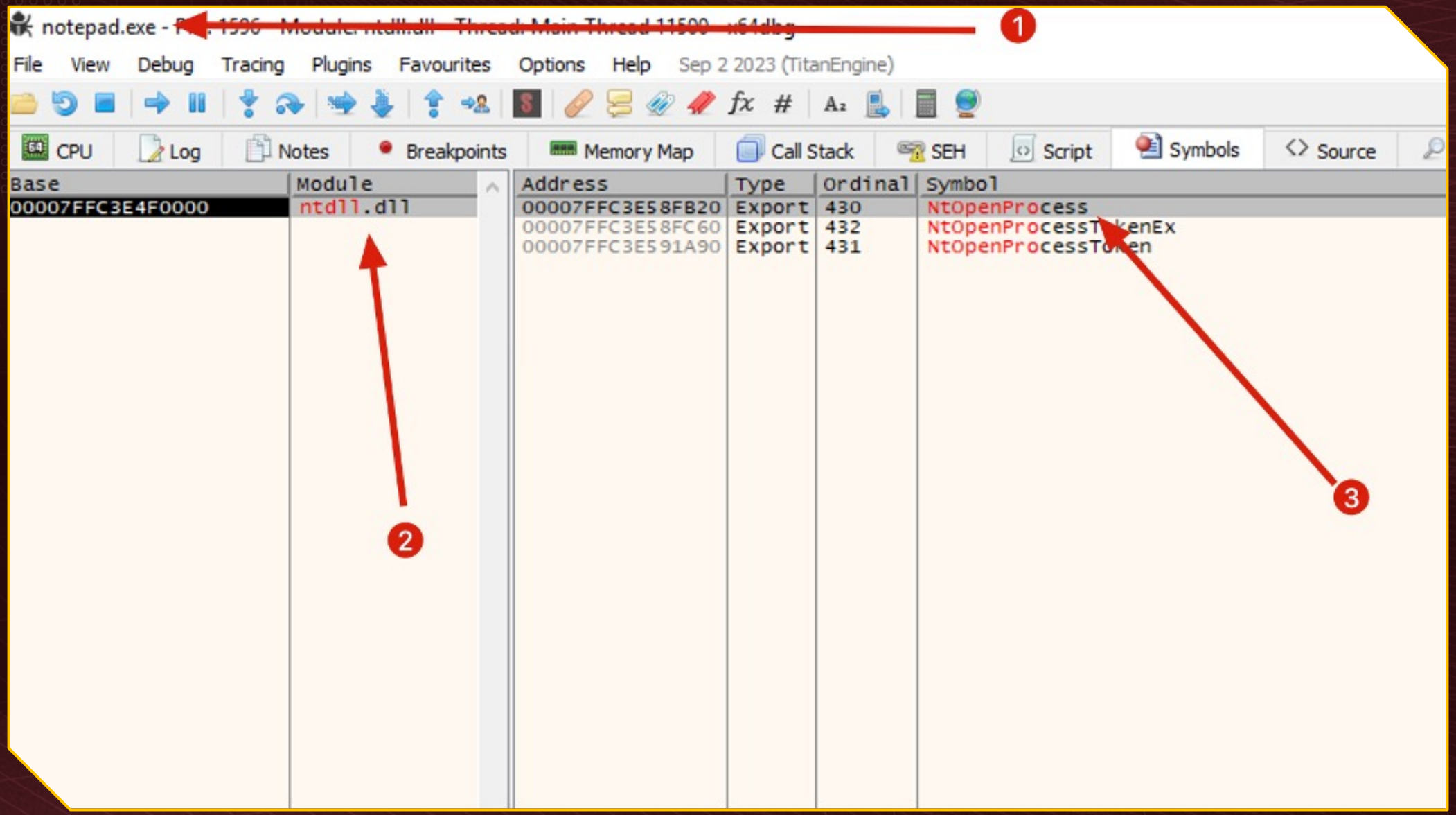
## IAT Hooked execution flow



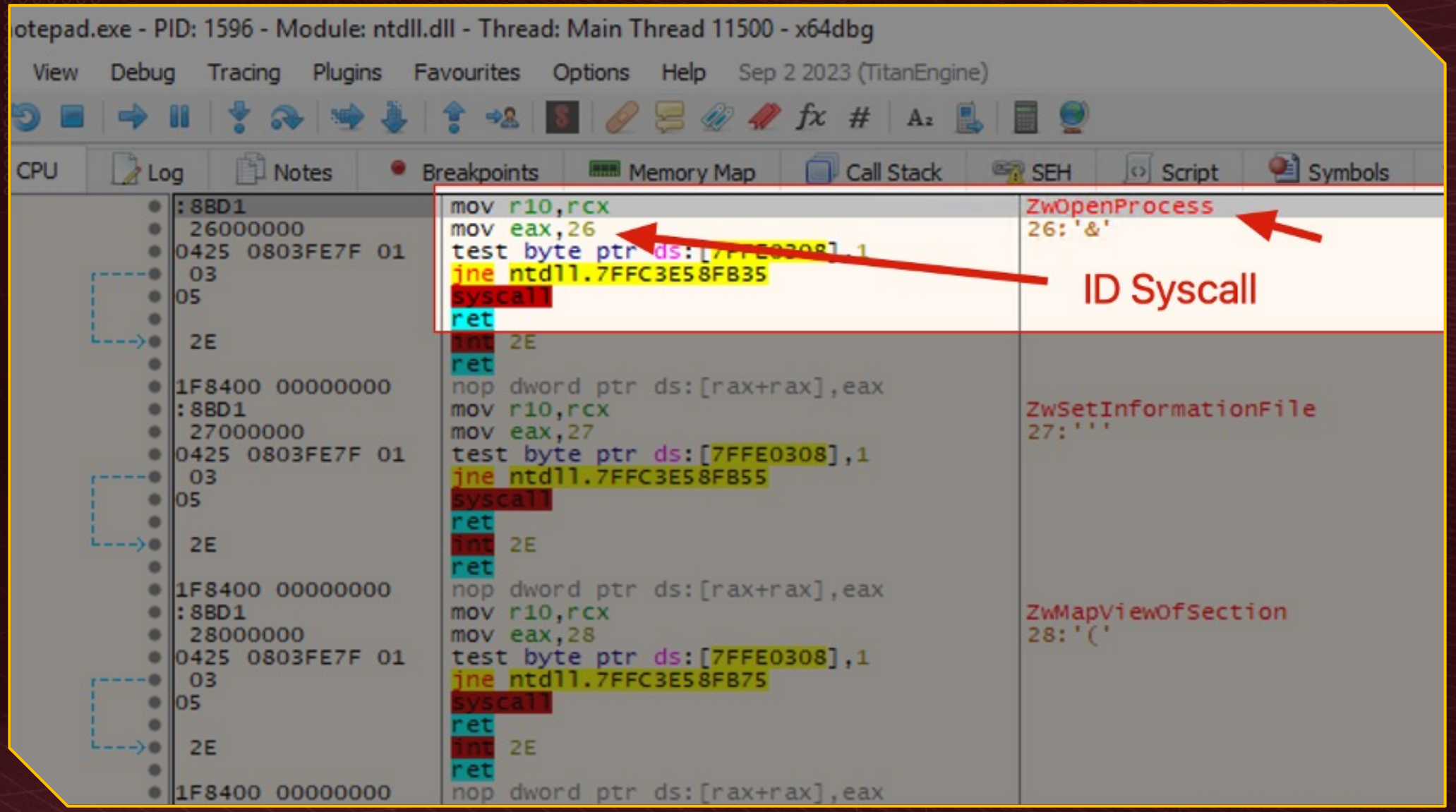
# EDR Overview



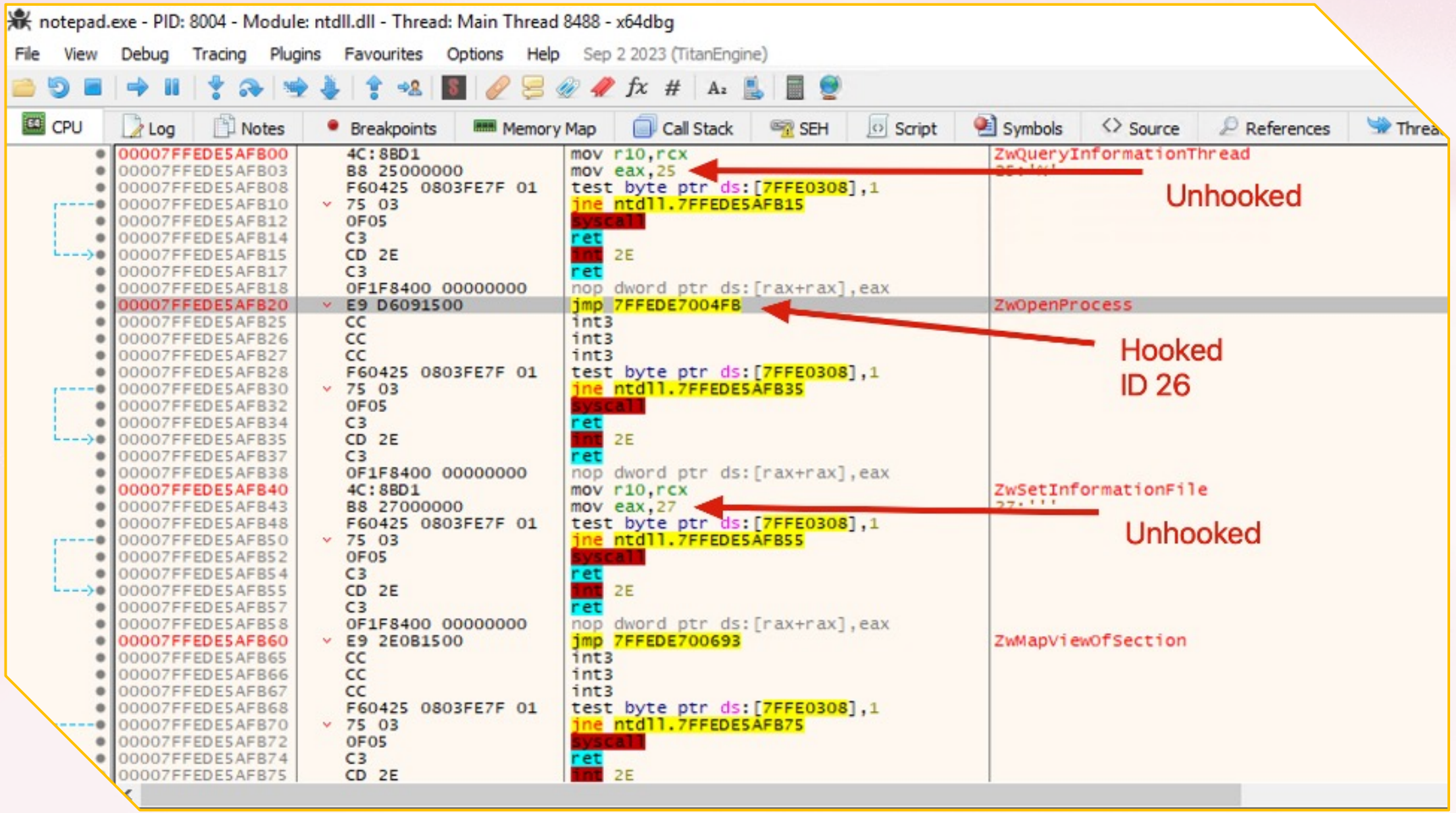
# Unhooked API



# Unhooked API



# Hooked API



# Most Common Bypasses

Remapping of Ntdll.dll

Direct Syscalls

Indirect Syscall

# Known techiniques

| | |
|---|---|
| Heaven’s Gate | Developed by Roy G. Biv |
| Hell’s Gate | Developed by smelly__vx (@RtlMateusz) and am0nsec (@am0nsec) |
| Halo's Gate | Developed by Reenz0h from Sektor7 |

# Known techiniques

| | |
|---|---|
| Heaven’s Gate | Developed by Roy G. Biv |
| Hell’s Gate | Developed by smelly__vx (@RtlMateusz) and am0nsec (@am0nsec) |
| **Halo's Gate** | **Developed by Reenz0h from Sektor7** |

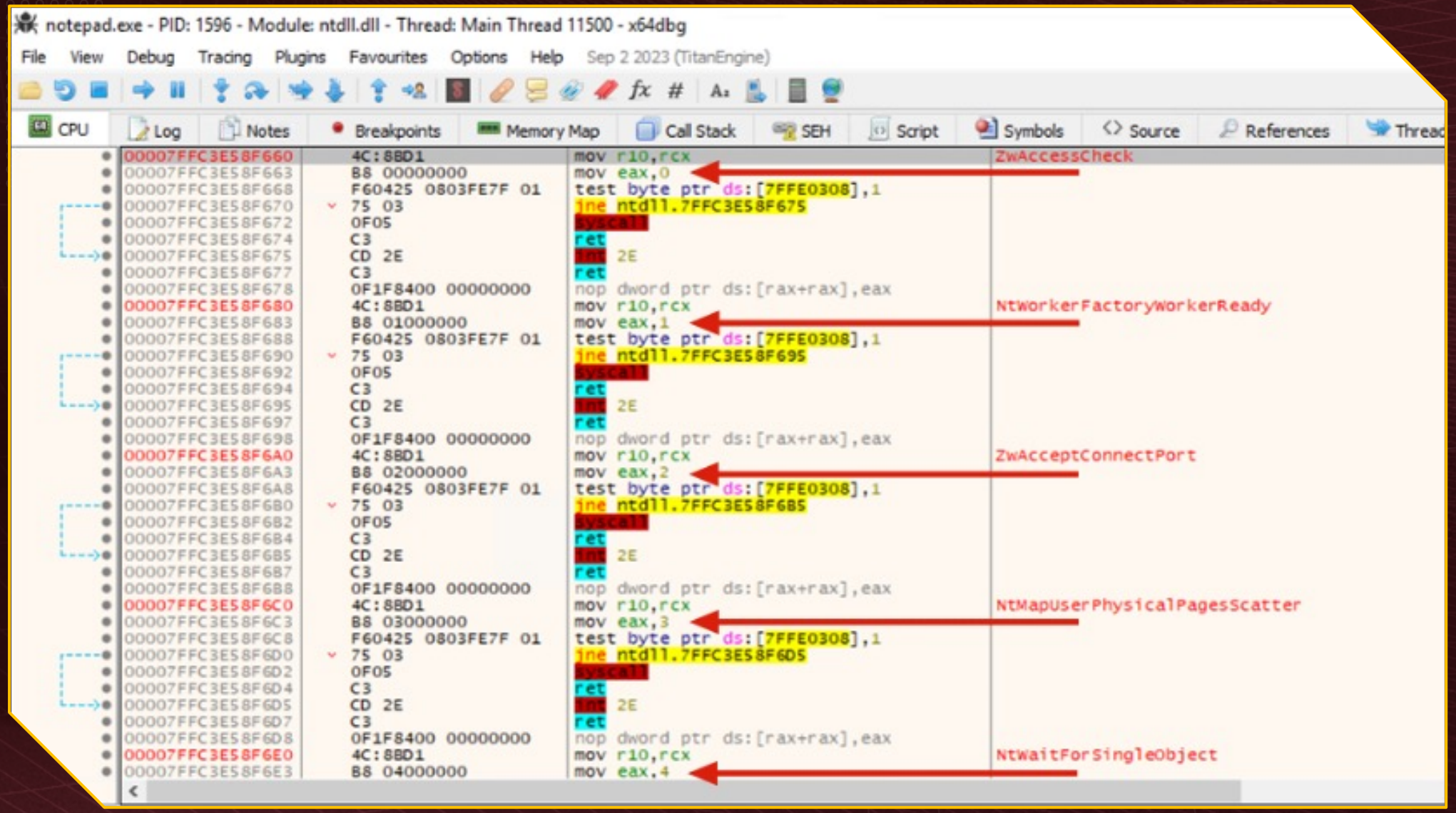
Locates the current address of the desired function within the Ntdll.dll


Performs the reading of the function's bytes (currently 32 bytes) and checks if the function's bytes match those of the assembly instructions (mov r10, rcx; mov eax, SSN)

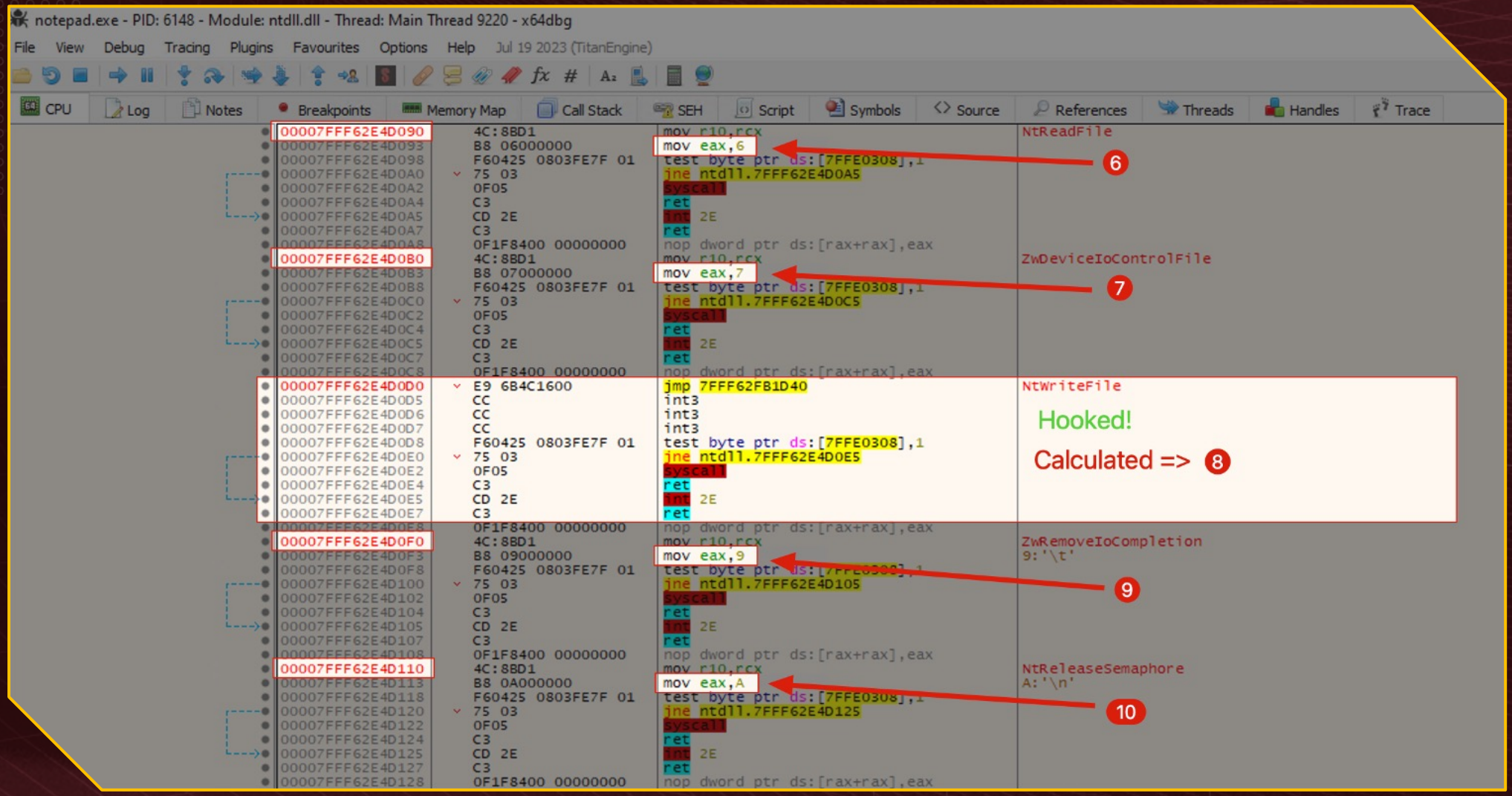
If these are not the bytes, the function is being monitored (in other words, it has a Hook set), however, the neighboring functions (before and after) may not have a Hook.

Searches in the neighboring functions (above and below) for functions without a Hook, and calculates the distance of the located function from the current function, thus having the current function's SSN code.

# System Service Number - SSN

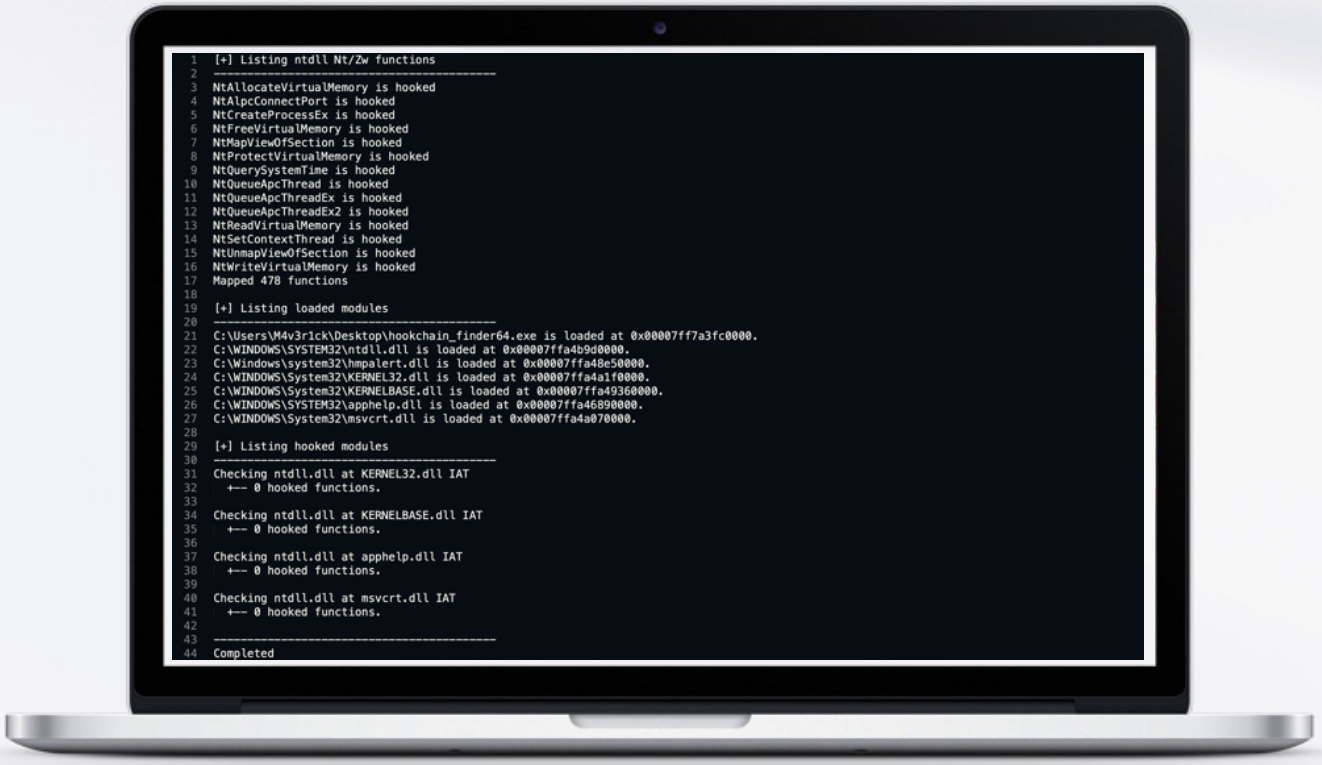


# System Service Number – SSN (Hallo's Gate)



# HookChain - Viability analysis

Preliminary feasibility, effectiveness, and scope verification of the HookChain technique




Code to just check if there are:

- Ntdll hooks
- IAT hooks

# Viability analysis

**Result 1:** 94% of the analyzed EDR solutions (15 out of 16) do not present hooks in the subsystem layer above Ntdll.dll, meaning, in the verification of all DLLs loaded in the application that reference Ntdll, only one EDR solution showed a hook in the IAT.

**Result 2:** 50% of the analyzed EDR solutions (8 out of 16) show an absence of hooks in user mode.

| PRODUCT | INTERCEPTION POINT (HOOK) | |
|---|---|---|
| | NTDLL | KERNELBASE / KERNEL32 |
| BitDefender | ✅ | ⛔ |
| CarbonBlack | ✅ | ⛔ |
| Checkpoint | ✅ | ⛔ |
| Cortex | ⛔ | ⛔ |
| CrowdStrike Falcon | ✅ | ⛔ |
| Windows Defender | ⛔ | ⛔ |
| Windows Defender + ATP | ⛔ | ⛔ |
| Elastic | ⛔ | ⛔ |
| ESET | ⛔ | ⛔ |
| Kaspersky | ⛔ | ⛔ |
| MalwareBytes | ⛔ | ⛔ |
| SentinelOne | ✅ | ✅ |
| Sophos | ✅ | ⛔ |
| Symantec | ⛔ | ⛔ |
| Trellix | ✅ | ⛔ |
| Trend | ✅ | ⛔ |

Enumeration carried out between March 1st and March 22nd, 2024.

SEC4US

System Service Number (SSN) dynamic mapping


Create a relationship table with SSN + Custom stub function of critical Ntdll and Hooked functions

Implant IAT Hook at all DLL subsystems that points to Ntdll critical and hooked functions

Continue to desired actions as usual

# HookChain implant flow

1. SSN - System Service Number mapping

Use of one of the dynamic mapping techniques of the SSN presented previously, such as Halo’s gate.

# HookChain implant flow

1. SSN - System Service Number mapping
2. Map critical functions

Mapping of some base functions for use in the actions of the next steps, such as:

a. NtAllocateReserveObject
b. NtAllocateVirtualMemory
c. NtQueryInformationProcess
d. NtProtectVirtualMemory
e. NtReadVirtualMemory
f. NtWriteVirtualMemory

# HookChain implant flow

1. SSN - System Service Number mapping
2. Map critical functions
3. Create and populate SYSCALL_LIST array

Creation and filling of an array where each item contains the following content:

a. SSN (Syscall Number)
b. Function address in Ntdll.dll
c. Memory address of the nearest SYSCALL instruction to the function in Ntdll.dll.

# HookChain implant flow

1. SSN - System Service Number mapping
2. Map critical functions
3. Create and populate SYSCALL_LIST array
4. DLL pre-loading

Preloads other DLLs, if it is known that the application in execution will dynamically load and use another DLL that has not yet been loaded in the current process, as well as verifying that this DLL to be loaded makes calls to functions of the Ntdll.dll.

# HookChain implant flow

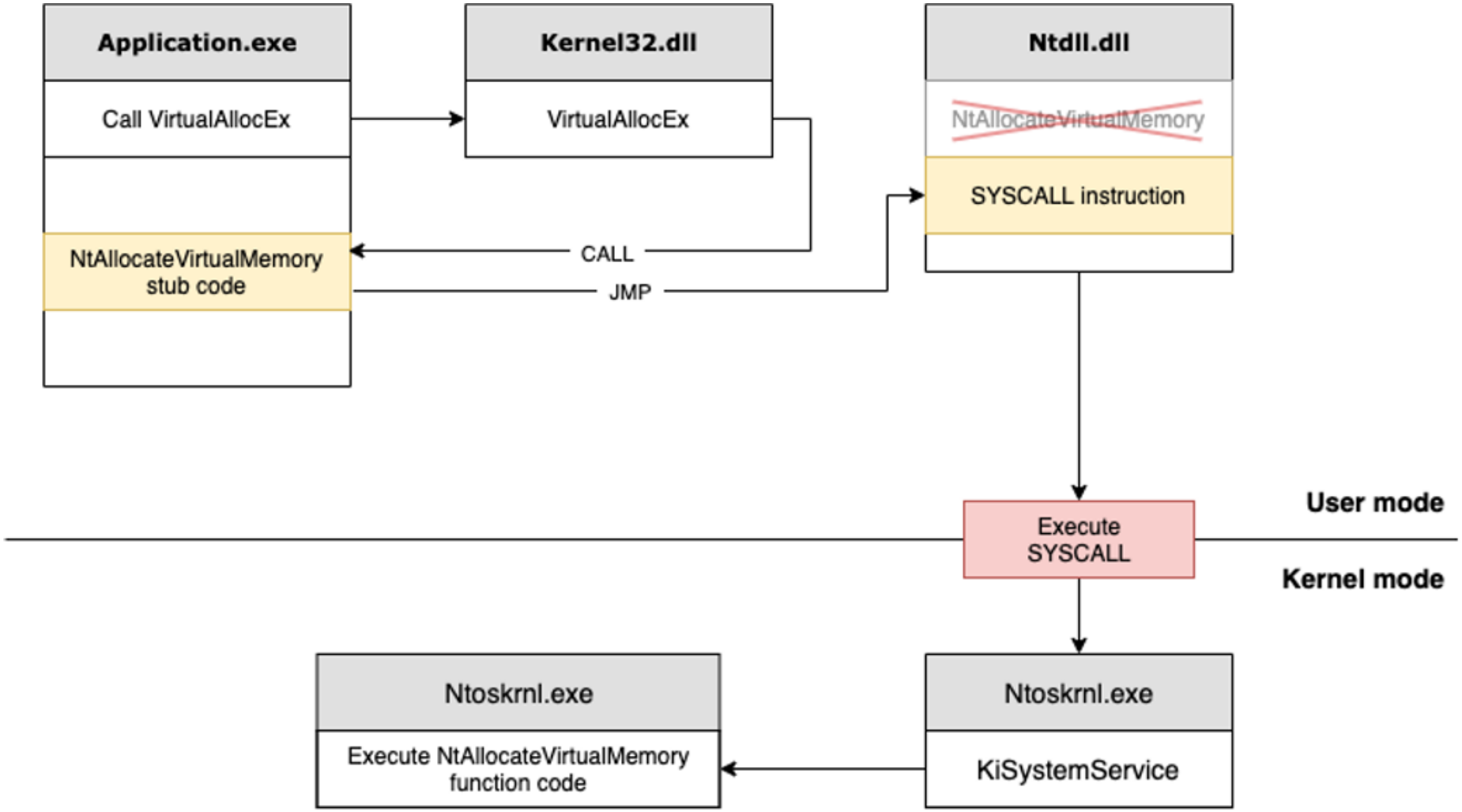
1. SSN - System Service Number mapping
2. Map critical functions
3. Create and populate SYSCALL_LIST array
4. DLL pre-loading
5. Implant IAT Hook

- Utilization of Indirect Syscall + mapped critical functions
  - Enumeration, reading and handling of the structures of the export and import tables of loaded DLLs.

- Modification of the IAT of key DLLs that use calls to Ntdll.dll such as:
  - kernel32, kernelbase, bcrypt, bcryptPrimitives, gdi32, mswsock, netutils, and urlmon.

# Execution Flow After HookChain Implant

## Simplified version



# Data structures and tables

Struct SYSCALL_INFO

```
typedef struct _SYSCALL_INFO {
    DWORD64 dwSsn;
    PVOID pAddress;
    PVOID pSyscallRet;
    PVOID pStubFunction;
    DWORD64 dwHash;
} SYSCALL_INFO, * PSYSCALL_INFO;
```

**dwSsn:** System Service Number.

**pAddress:** Virtual address of the function within Ntdll.

**pSyscallRet:** Virtual address of a SYSCALL instruction within Ntdll.

**pStubFunction:** Virtual address of the HookChain interception (implant) function, this is the address to which all calls to the function in question will be directed. In other words, this is the address that will be assigned in the IAT in replacement of the virtual address of the Ntdll function.

**dwHash:** Function identification hash. This hash is calculated through the name of the ntdll function. The function name is not stored and used to make identification by EDRs more difficult.

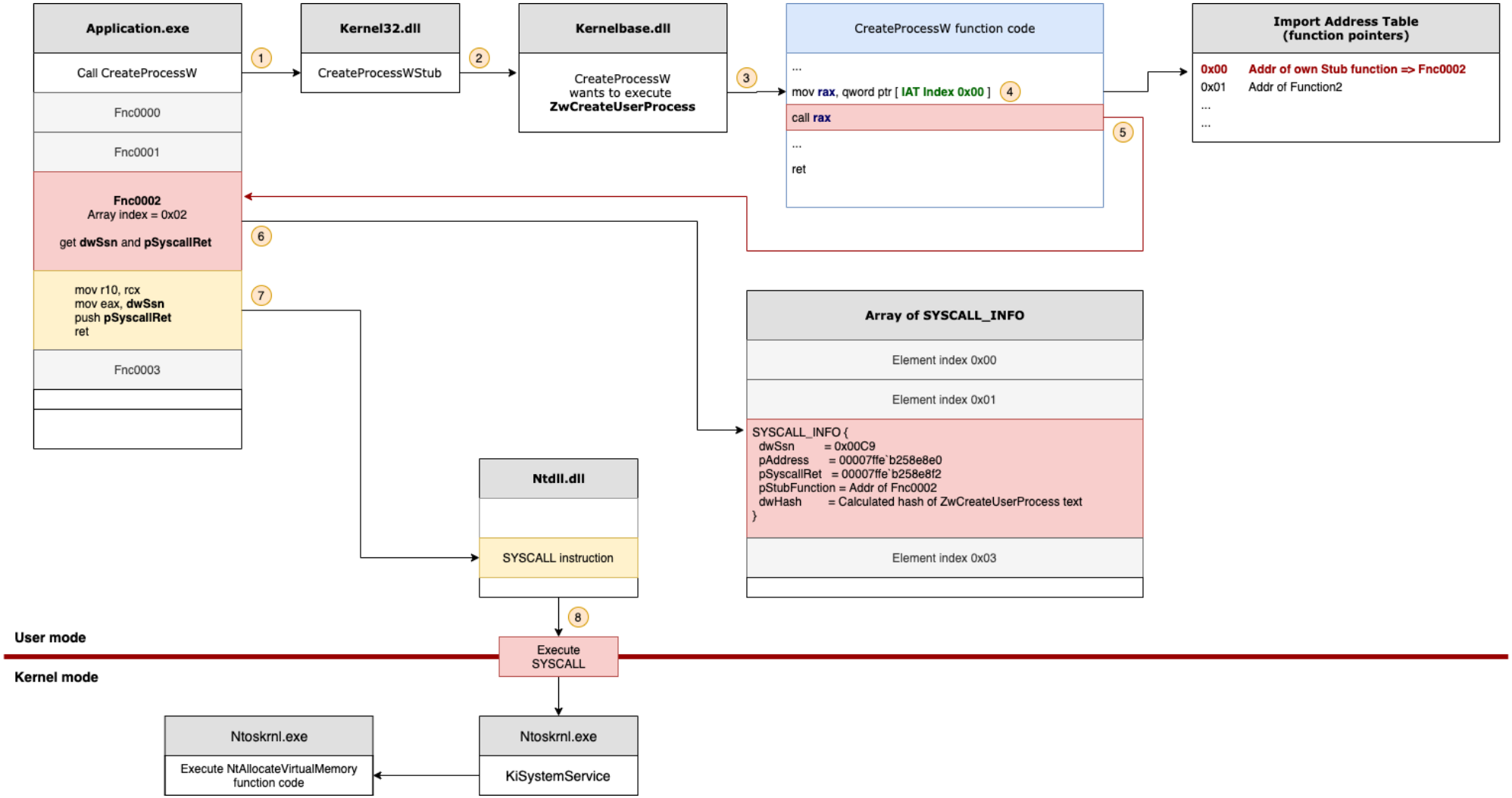
# Data structures and tables

Struct SYSCALL_LIST

```
typedef struct _SYSCALL_LIST
{
    DWORD64 Count;
    SYSCALL_INFO Entries[MAX_ENTRIES];
} SYSCALL_LIST, * PSYSCALL_LIST;
```

Array of SYSCALL_INFO

# Execution Flow After HookChain Implant



# Demo 1

Understanding the flow in a debug environment

# Advantages of HookChain

- Reduction in the probability of identification by the EDR due to following the rules proposed by some telemetries such as:
- Total execution time of the process, as the execution time of the calls will remain very close to the original process.
- Execution chain, where the EDR expects the function call to have come from the application, then passed through Kernel32.dll, then through Ntdll.dll. This is due to the fact that the HookChain implant (interception function) passes transparently in the call stack.
- No effort and/or modification necessary for the execution of pre-existing codes/applications as the interception occurs broadly and transparently for the application in execution.

# Demo 2

Non admin C2 (Command and Control) session

# Tests environment!

- 100% of tests had done using the same procedure in my own controlled environment.

- As the test focus is EDR/XDR protections the windows protections (RunAsPPL and Credential Guard) was disabled during the tests.

- Source of Virtual Machines and/or Licensing:
  - Bought by my own resources;
  - Trial licensing;
  - Provided by friends from his corporate environment;

Many thanks to my friends who supported me during this research. Without you, this would not have been possible.

# Demo 3

Execute Mimikatz

# Gartner Magic Quadrant - 2023

## Endpoint Protection Platforms

I was able to test 56% (9 out of 16) EDR/XDR products present in Gartner 2023 report.

In additional, I tested other 4 products not present in Gartner report, Acronis, Cylance, Elastic and MalwareBytes.

Tested products marked in green.

# Result table

| PRODUCT | EXECUTED CODE | | | | | | |
|---|---|---|---|---|---|---|---|
| | Remote Process Injection | Download, local PE injection and executing | | | | | |
| | | Meterpreter | Havoc | Meterpreter + Kiwi | Mimikatz | Procdump | LSASS Dump |
| Acronis | ⚠️ | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |
| BitDefender | ✅ | ⛔ | ✅ | ⛔ | ⚠️ | ⛔ | ⚠️ |
| Cortex | ⛔ | ⛔ | ✅ | ⛔ | ⛔ | ⛔ | ✅ |
| CrowdStrike Falcon | ✅ | ✅ | ✅ | ⛔ | ⚠️ | ⚠️ | ✅ |
| Cylance | ⛔ | ⚠️ | ⛔ | ⛔ | ✅ | ⛔ | ⛔ |
| Windows Defender | ✅ | ⚠️ | ✅ | ⛔ | ⛔ | ⛔ | ✅ |
| Windows Defender XDR | ✅ | ✅ | ✅ | ⛔ | ⛔ | ⛔ | ✅ |
| Elastic | ✅ | ⛔ | ⛔ | ⛔ | ⛔ | ⛔ | ⛔ |
| ESET | ✅ | ⚠️ | ✅ | ✅ | ✅ | ⛔ | ✅ |
| MalwareBytes | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |
| SentinelOne | ⚠️ | ✅ | ⚠️ | ✅ | ✅ | ⚠️ | ⚠️ |
| Sophos | ✅ | ✅ | ✅ | ⚠️ | ⚠️ | ⛔ | ⚠️ |
| Trellix | ✅ | ✅ | ✅ | ✅ | ✅ | ⛔ | ✅ |
| Trend | ✅ | ✅ | ✅ | ✅ | ✅ | ⛔ | ✅ |

✅ Executed without alerts and blocks.

⚠️ Partially execution (no success) without alerts or Executed (successfully) with alerts.

⛔ Execution fail with block and alert

SEC4US

I’m just a child who has never grown up. I still keep asking these 'how' & 'why' questions. Occasionally, I find an answer.

*Stephen Hawking*

https://github.com/helviojunior/hookchain/

- Full white paper
- Source code
- References


www.sec4us.com.br